

Week 1 Overview: OpenStack, Terraform, Ansible

Summary

Week 1 takes us from an empty Sebowa project to a **reproducible five-node Linux environment**. Nothing in this week involves Kubernetes or agents. The purpose is to build a foundation that is solid enough to build on, and that we can destroy and recreate from code.

Creating five virtual machines is **the easy part**. The real objective is that **every team member understands**:

- what OpenStack is responsible for,
- what Terraform is responsible for,
- what Ansible is responsible for,
- what is secret and must never enter Git,
- how to prove that each layer works on its own.

Conceptual Understanding

Each tool answers a different question, and the week follows them in order.

LAYER	QUESTION IT ANSWERS	EXAMPLE
OpenStack	What cloud do we have to work with?	Project, quota, images, flavors, networks
Terraform	What infrastructure should exist?	Networks, router, security groups, five VMs, one public IP
Ansible	How should the machines be configured?	Packages, time sync, WireGuard, DNS, firewall, HAProxy

The chain we should be able to explain without notes by Friday:

OpenStack project and quota

- > networks, router, ports, security groups
- > VMs (and storage where configured)
- > Terraform state and desired infrastructure
- > SSH through the edge node
- > Ansible inventory
- > Linux host configuration
- > edge services and the HAProxy API endpoint

Two points are worth remembering:

- A VM showing **ACTIVE** in OpenStack does not prove Linux is reachable.
- An Ansible run that finishes without errors does not prove a service is reachable through the intended path.

Every layer has its own test, and we run that test before moving on.

What is Built

A student-sized version of the reference platform, with five hosts:

HOST	VCPU	RAM	STORAGE	ROLE
edge-01	4	10 GiB	50 GiB	Bastion and VPN, DNS, host firewall (security services arrive in Week 3)
api-lb-01	2	4 GiB	25 GiB	HAProxy; the stable Kubernetes API endpoint
k8s-cp-01	4	8 GiB	30 GiB	Kubernetes control plane and cluster-admin host
k8s-worker-01	8	16 GiB	40 GiB	Platform and project workloads
k8s-worker-02	8	16 GiB	40 GiB	Platform and project workloads

Together this uses roughly 26 vCPU and 54 GiB RAM, so we **confirm our quota has enough headroom before applying anything.**

Access works in two stages. At first we reach `edge-01` through a single floating IP over SSH. By the end of the week the intended admin path is **WireGuard through the edge node**, and public SSH is only a bootstrap and recovery step.

Where does What Happen?

Authoring infrastructure and administering the cluster are kept deliberately separate.

LOCATION	TOOLS AND TASKS
Workstation	Git, SSH, OpenStack CLI, Terraform, Ansible, kubeseal
<code>edge-01</code>	WireGuard, DNS, nftables, host diagnostics
<code>api-lb-01</code>	HAProxy and load balancer diagnostics
<code>k8s-cp-01</code>	kubectl and cluster diagnostics (from Week 2)
GitHub	Pull requests, CI, image builds, desired state
Kubernetes	Argo CD and everything it manages (later weeks)

Two things to remember from this:

- We do **not** copy a Kubernetes admin kubeconfig to a personal laptop.
- We do **not** make permanent application changes with `kubectl edit`. We diagnose on `k8s-cp-01` and fix the desired state in Git.

The week in four phases

The Nyameko's guide lists fifteen steps. They group naturally into four phases.

Phase 1: Prepare and understand the cloud

- Start from the `dev` branch and create a feature branch (`feature/week1-foundation`). Never work directly on `main` or `dev`.
- Check the workstation tools are installed: Git, SSH, OpenStack CLI, Terraform, Ansible, kubeseal.
- Create private local folders for credentials, outside the repository, and set up `.gitignore` so state files, keys, and credentials can never be committed.
- Authenticate with `openstack token issue`. If this fails, stop and fix it; Terraform cannot repair broken authentication.
- Check project quota, then list the images, flavors, networks, volume types, and security groups. We inspect what the provider gives us rather than guessing names or IDs.

- Agree on the network and trust design, including the assigned CIDR ranges.

Phase 2: Terraform builds the infrastructure

- `terraform init`, `fmt`, and `validate` to confirm the configuration is coherent.
- `terraform plan -out=tfplan`, then read the plan together as a team. We should be able to account for every resource: two private networks, one router, the security groups, exactly five hosts, and one edge floating IP.
- Treat any `-/+` in a plan as a warning, because it means a resource will be destroyed and recreated.
- `terraform apply tfplan`. We do not use `-auto-approve`; reviewing the plan is part of the learning.
- Verify the result **independently of Terraform** with the OpenStack CLI, then run `terraform plan` again and expect no changes.

Phase 3: Ansible configures the machines

- SSH to `edge-01` and confirm it behaves as expected. If SSH times out, check in this order: floating IP, security group, guest routing, guest firewall, `sshd`. We do not switch off SELinux or the firewall to make a problem go away.
- Build a private Ansible inventory (only a template is committed) and inspect what Ansible actually resolves with `ansible-inventory --graph` and `--host`.
- Run `ansible all -m ping`. Every host must be reachable before roles are applied.
- Syntax-check, then run the **base bootstrap** on all five hosts: packages, time synchronisation, timezone, admin and SSH setup.
- Run the bootstrap a second time. A well-written playbook should report close to `changed=0`.
- Configure the **edge subset**: WireGuard, DNS (Pi-hole), and the nftables firewall. Wazuh and Suricata are left for Week 3 on purpose.

Phase 4: Secure access and the API endpoint

- Bring up WireGuard from the workstation and confirm we can SSH to the private hosts through it.
- **Only after** the VPN path is proven, reduce public SSH exposure, and do it through Terraform (plan, review, apply) rather than by editing security groups in Horizon.

- Configure **HAProxy** on `api-lb-01` as the stable Kubernetes API endpoint, pointing at `k8s-cp-01:6443`. It is normal for the backend to show as DOWN until Kubernetes is installed in Week 2.
- Validate the HAProxy config and service. If it fails to bind, check SELinux before disabling it; the upstream deployment needed the `haproxy_connect_any` boolean on Rocky Linux.

Key points to carry through the week

- **OpenStack security groups and nftables are different things.** This we know from SCC. The security group filters traffic before it reaches the VM; nftables filters inside the VM. Both must allow a path for it to work.
- **Secrets stay out of Git.** This includes `terraform.tfstate`, `clouds.yaml`, RC files, SSH and WireGuard private keys, and application credentials. The WireGuard server private key stays on `edge-01` and each client private key stays on its client; only public keys are exchanged.
- **Timezone and time synchronisation are separate.** The timezone only changes how time is displayed; Chrony keeps the clock correct. This matters for correlating security events in Week 3.
- **Use explicit container registries** for images launched by system services, because short image names can fail to resolve in non-interactive Podman services.
- **Follow the order.** Do not skip ahead; later steps depend on earlier ones being stable.

Week 1 Objectives

LAYER	EVIDENCE	EXPECTED RESULT
Git	Branch and PR model	Work done on a feature branch from <code>dev</code>
OpenStack auth	<code>openstack token issue</code>	Succeeds
Quota	<code>openstack quota show</code>	Enough headroom
Terraform	<code>fmt</code> , <code>validate</code> , reviewed plan	Clean
Terraform drift	Second <code>terraform plan</code>	No unexpected changes
Nova and Neutron	Server, port, router, security group lists	Match the diagram

LAYER	EVIDENCE	EXPECTED RESULT
Public exposure	Floating IP list	Only the intended edge path
SSH	Edge and private access	Works through the intended path
Ansible inventory	<code>--graph</code> and <code>--host</code>	Correct grouping and variables
Ansible convergence	Second bootstrap run	No unexplained recurring changes
WireGuard	Private SSH	Works before public SSH is removed
HAProxy	Syntax, service, listener	Healthy; backend may be DOWN until Week 2
GitHub	Week 1 PR	Reviewed and merged into <code>dev</code>

Exit gate

- Terraform can rebuild the five-node topology.
- Ansible can reach and configure all five hosts.
- The edge provides a verified private administration path.
- HAProxy provides the stable API endpoint and is ready for Kubernetes.
- The team can explain every boundary above without needing the instructor to diagnose it.

Evidence and pull request

We keep a small `evidence/week1/` index, or a PR checklist, pointing to sanitised evidence:

- architecture and trust-boundary diagram,
- Terraform provider version lock and Git SHA,
- sanitised plan summary,
- OpenStack resource summary,
- Ansible inventory graph,
- first and second bootstrap recaps,
- WireGuard and private-access proof with secrets omitted,
- HAProxy validation,
- a short note on at least one real problem we hit and how we solved it.

Before pushing, run `git status`, `git diff --check`, and `git diff --cached`. Then commit, push the feature branch, and open a pull request into `dev`.

Debugging

SYMPTOM	CHECK FIRST
OpenStack CLI returns 401 or 403	Credentials, project, and role; this is not a Terraform problem
Terraform cannot find a flavor or image	The provider's exact ID or name, and the region
VM is ACTIVE but SSH times out	Floating IP, then security group, route, nftables, <code>sshd</code>
Ansible variable seems missing	<code>ansible-inventory --host <name></code>
HAProxy config is valid but the service fails	Journal, SELinux, bind address and port
Private host unreachable after VPN is up	WireGuard route, OpenStack router return route, security group
